

EchoMind面试面经2（人员已满，不可申请）

⚽ 大家如果拿这个项目面试了之后一定要在后台私信我把问题发我，我会把面试题和答案都给总结出来，同时根据面试问题优化EchoMind项目。希望大家可以一起进步！

当前面经已覆盖字节，华为，美团，Momenta等公司面试问题，从各种角度对echomind进行拷打。如果大家非常着急面试一定要先看面经！看问得是哪些问题！

1. 企业智能客服多智能体系统的主要目标是什么？

EchoMind 的主要目标是做一个面向企业客服场景的智能客服系统，不只是简单调用大模型聊天，而是把客服链路里比较关键的能力工程化，包括意图识别、多 Agent 路由、知识库增强、多轮记忆、监控和评测。

从用户请求进入 /chat 开始，系统会先读取用户的多轮记忆，再判断用户意图，同时检索业务知识库，把记忆和知识库内容一起注入到 Agent 上下文中，最后由合适的 Agent 生成回复。回复完成后，系统还会把对话写回记忆，并异步更新用户画像。

所以它的目标可以概括为三点：

- 提高客服回答准确性：通过 RAG 知识库约束模型回答，减少幻觉。
- 提高问题处理效率：通过意图识别把技术、账单、通用问题路由到不同 Agent。
- 提高系统可运营性：通过 Monitor 和评测框架观察 Agent 表现，发现退化并持续优化。

2. 可以介绍一下 Agent 的编排吗？具体有哪几个模块？

🍌 Agent 编排的核心在 `AgentOrchestrator`。它负责接收用户请求，判断意图，选择 Agent，执行回复，并处理失败降级和人工升级。

主要模块包括：`Request` 请求对象、`IntentRecognizer` 意图识别器、Agent 类型枚举、Agent Pool、路由逻辑、并行协作逻辑和 `AgentStats` 运行统计。

整体流程是：先识别意图，再根据意图选择 Technical、Billing 或 General Agent；如果是复合问题，比如登录报错同时重复扣款，会并行调用多个 Agent；如果专属 Agent 失败，会降级到 GeneralAgent。

另一个版本：

Agent 编排的核心在 `agents/agent_orchestrator.py`，入口是 `AgentOrchestrator.run()`。它负责把一个用户请求从“原始文本”变成“由哪个 Agent 处理、是否需要并行协作、是否需要升级”的完整决策。

主要模块有：

- Request：编排器输入，包含用户消息、用户 ID、会话 ID、记忆和知识库拼成的上下文、最近对话历史。
- IntentRecognizer：意图识别模块，用来判断用户是技术问题、账单问题、投诉、问候还是转人工等。
- AgentType：Agent 类型枚举，包括 GENERAL、TECHNICAL、BILLING、ESCALATION。
- BaseAgent：所有 Agent 的基类，封装 LLM 调用、统计指标、异常处理和升级关键词判断。
- GeneralAgent、TechnicalAgent、BillingAgent：具体业务 Agent。
- Agent Pool：按 Agent 类型维护实例池，当前每类默认一个实例，但代码支持同类多实例。
- 路由逻辑：包括意图路由、性能路由、失败降级和复合问题并行协作。
- AgentStats：记录每个 Agent 的调用次数、成功率、平均延迟和 Monitor 写回的惩罚项。

整体流程是：

代码块

```
1 Request
2   -> 意图识别
3   -> 判断是否复合问题
4   -> 选择 Agent 类型
5   -> 从 Agent Pool 中选择 routing_score 最高的实例
6   -> 调用 Agent
7   -> 失败时降级到 GeneralAgent
8   -> 判断是否需要升级人工
9   -> 返回 OrchestratorResult
```

3. 这 3 个 Agent 在实现方式上有什么区别？

 这 3 个 Agent 底层实现是统一的，都继承自 `BaseAgent`，都通过同一个 LLM 客户端生成回复。

主要区别是职责和 system prompt 不同。`GeneralAgent` 负责通用咨询和兜底；`TechnicalAgent` 负责故障排查、登录失败、报错这类技术问题；`BillingAgent` 负责退款、扣款、发票、订阅这类账单问题。

另外系统会分别记录它们的成功率和延迟，方便后续监控和路由优化。

另一个版本

这 3 个 Agent 都继承自 BaseAgent，所以底层实现方式基本一致，都是调用同一个大模型客户端，只是各自的 agent_type 和 system_prompt 不同。

具体区别是：

- GeneralAgent：通用客服，负责普通咨询、兜底回复和无法明确分类的问题。
- TechnicalAgent：技术支持，专注故障排查、错误诊断、系统配置，例如登录失败、500 错误、应用崩溃。
- BillingAgent：账单服务，专注退款、扣款、发票、订阅、账户相关问题。

它们不是三个完全不同的模型，也不是三个完全不同的服务进程。当前实现里它们的差异主要体现在角色提示词、路由入口和统计维度上。这样设计的好处是实现简单、可控，同时后续可以很自然地扩展成不同模型、不同工具权限或不同知识库权限。

4. 为什么需要 3 个 Agent 做多 Agent 路由？它们是提示词不一样，还是有别的差异？



当前实现里，最直接的差异确实是提示词和职责边界不同，但多 Agent 的价值不只是换提示词。

客服问题类型差异很大。技术问题需要排查步骤，账单问题需要更谨慎，通用问题则适合简洁回答。拆成多个 Agent 后，路由更清晰，监控也能分开统计。

同时这个架构方便后续扩展。现在主要是 prompt 不同，后续可以给不同 Agent 配不同工具、知识库、模型或权限。

另一个版本：

当前版本里，3 个 Agent 的核心差异主要是提示词和职责边界不同，同时系统还为它们维护了独立的运行统计和路由策略。

需要多 Agent 的原因是客服场景的问题类型差异比较明显。技术问题需要排查步骤，账单问题需要谨慎处理退款和发票，通用问题则更适合简洁回答。如果所有问题都交给一个通用 Agent，提示词会变得很宽泛，容易出现回答不聚焦、处理策略不一致的问题。

多 Agent 的价值主要有：

- 职责隔离：技术、账单、通用问题采用不同回答风格和处理原则。
- 路由可控：通过意图识别把请求分配给最合适的 Agent。

- 监控可拆分：可以单独观察 technical、billing、general 的成功率和延迟。
- 可扩展：后续可以让不同 Agent 绑定不同工具、不同知识库、不同模型或不同权限。
- 支持协作：复合问题可以并行派发给多个 Agent，例如“登录报错且重复扣款”会同时调用技术和账单 Agent。

所以，当前代码里最直接的差异是提示词，但架构上已经为更深层的差异预留了空间。

5. 三路融合规则识别器可以讲一下吗？它的目的是什么？

🏐 三路融合识别器在 `core/intent_recognizer.py`，目的是判断用户当前消息的客服意图，比如技术问题、账单问题、投诉、查询或转人工。

它融合了三路信号：第一路是 LLM 语义理解，负责复杂表达和上下文；第二路是向量相似度，把用户问题和意图模板做匹配；第三路是关键词规则，覆盖退款、报错、转人工等高频表达。

最后三路结果会做加权投票。这样比单纯关键词更准确，也比只依赖 LLM 更稳。

另一个版本：

三路融合意图识别器在 `core/intent_recognizer.py`，目的是判断用户消息属于哪类客服意图，例如查询、投诉、请求、技术问题、账单问题、账户问题、转人工等。

它不是简单的关键词分类，而是融合三种策略：

- LLM 语义理解：负责理解复杂表达和上下文，是主判断依据。
- 向量相似度：把用户消息和各类意图模板做相似度匹配，用来补充常见表达。
- 关键词 Pattern：本地规则兜底，例如“退款”“发票”“报错”“转人工”等高频词。

这样做的目的有两个：

- 提高准确性：LLM 能理解复杂语义，向量和规则能补充稳定信号。
- 提高鲁棒性：当 LLM 失败或兼容 API 不支持 embedding 时，系统仍然有兜底判断。

在官方 API 模式下，权重是 LLM 70%、向量 20%、关键词 10%。如果配置了第三方兼容接口，系统会默认禁用远端 embedding 分支，变成 LLM 85% + 关键词 15%。

6. 三路融合识别器的输出是什么？是最终意图吗？

👉 它输出的是 `IntentResult`，里面包含最终意图，但不只有意图。

核心字段是 `intent`，这是三路融合后的最终意图，会用于 Agent 路由。除此之外还有 `confidence` 置信度、`urgency` 紧急程度、`entities` 实体信息、`reasoning` 判断理由和 `latency_ms` 耗时。

所以它既是分类结果，也是后续路由和处理的结构化输入。

7. 你如何通过向量相似度和关键词规则来判断意图？

❤️ 向量相似度这一路是把用户输入和预设的意图模板做匹配。比如 `technical` 有“应用崩溃”“无法登录”“500 错误”，`billing` 有“申请退款”“重复扣款”“发票问题”。系统计算用户输入和这些模板的余弦相似度，得到一个候选意图。

关键词规则更直接，比如出现“退款、扣款、发票”倾向 `billing`，出现“报错、崩溃、error”倾向 `technical`，出现“转人工、经理、投诉”倾向 `escalation` 或 `complaint`。

最后这两路不会单独决定结果，而是和 LLM 结果一起加权投票。

另一个版本

向量相似度这一路使用预定义的意图模板。每个意图都有一些典型表达，例如：

- `technical`：应用一直崩溃、无法登录、出现 500 错误。
- `billing`：为什么扣了两次款、申请退款、发票问题。
- `account`：修改邮箱、注销账户、更新个人信息。

系统会把模板文本转换成向量，用户输入也转换成向量，然后计算余弦相似度。哪个意图模板和用户输入最接近，就给哪个意图更高的相似度分数。

当前代码里有一个工程化兜底：如果客户端未来支持 `embeddings.create`，会优先使用远端 `embedding`；但当前 Anthropic SDK 没有这个资源，所以实际会退化为本地字符 `n-gram` 哈希向量。它不是严格语义 `embedding`，但可以稳定地提供相似表达匹配能力。

关键词规则这一路更直接，会维护一组高频关键词：

- 出现“退款”“扣款”“发票”“refund”，倾向 `billing`。
- 出现“崩溃”“报错”“error”“crash”，倾向 `technical`。
- 出现“转人工”“经理”“投诉”，倾向 `escalation` 或 `complaint`。
- 出现“密码”“邮箱”“账户”，倾向 `account`。

最后不会直接用某一路的结果，而是把 LLM、向量、关键词三路结果做加权投票，得分低于阈值则降级为 `other`。

8. 知识库里是否需要对所有检索知识做打标？



不需要对所有知识做人工意图打标。

当前 RAG 知识库的主要作用是回答业务问题，不是做意图分类。知识库文档导入时只需要提供 title 和 content，系统会自动切片并存入 ChromaDB。检索时通过语义相似度找到相关文档片段。

也就是说，知识库不要求每条文档标注“这是技术类”“这是财务类”。当然，如果后续希望做更精细的权限控制、知识域过滤或垂直路由，可以在 metadata 里增加 category、department、product 等标签。但这不是当前系统运行的必要条件。

当前系统里，意图识别主要由 IntentRecognizer 完成，知识库主要由 KnowledgeBase 用于 RAG 检索，两者职责是分开的。

另一个版本：

当前实现不需要对所有知识做人工打标。

知识库的作用是 RAG 检索，为最终回答提供业务知识，不是直接做意图分类。文档导入时只需要 `title` 和 `content`，系统会自动切片并存入 ChromaDB。

如果后续要做权限控制或按部门过滤，可以在 metadata 里加标签，但当前主链路不依赖这些标签。

9. 如果向量相似度检索出来的是知识库中的一条 document，它是怎么转换为意图的？



当前系统里，知识库 document 不会转换成意图。

这里有两条链路要区分。意图识别里的向量相似度，是用户问题和意图模板做匹配，用来辅助判断 intent。RAG 知识库里的向量检索，是用户问题和知识库文档做匹配，用来找回答所需资料。

所以检索到“退款政策”文档，只会作为上下文给 Agent 使用，不会直接变成 billing 意图。最终意图仍然来自 `IntentRecognizer`。

另一个版本

在当前实现里，知识库检索出来的 document 不会转换成意图。

这是一个容易混淆的点：EchoMind 里有两类“向量/相似度”能力：

- 意图识别里的向量相似度：用户问题和意图模板做相似度，用来辅助判断意图。
- RAG 知识库里的向量检索：用户问题和知识库文档做相似度，用来找到回答问题所需的知识片段。

知识库 document 的作用是作为上下文提供给最终生成 Agent，例如检索到“退款政策”，它会被拼进 [知识库检索结果]，让 Agent 按政策回答“退款多久能到账”。

最终意图仍然来自 IntentRecognizer 的三路融合结果，而不是由知识库 document 反推出意图。

10. 你是不是只有技术相关知识库和财务相关知识库，检索到哪个就算哪个意图？

 不是。当前项目只有一个统一的 `knowledge_base` collection，里面有退款政策、订单查询、账户安全、技术故障排查、会员积分、配送说明等文档。

意图判断不是靠检索到哪个知识库，而是靠 `IntentRecognizer` 三路融合输出 intent。

知识库负责“回答时参考什么内容”，意图识别负责“问题交给哪个 Agent”。这两个职责是分开的。

11. RAG 知识库构建这一块具体做了什么？

 AG 知识库在 `mcp/knowledge_base.py`。它主要做了文档导入、切片、向量化存储和语义检索。

系统启动时如果知识库为空，会导入默认客服文档。也支持通过 `/knowledge/add` 批量导入，或通过 `/knowledge/upload` 上传 txt、md、json 文件。

长文档会按大约 500 字切成 chunk，然后存入 ChromaDB 的 `knowledge_base` collection。查询时通过 ChromaDB 的 `query_texts` 返回最相关的 Top-K 文档片段。

这个知识库还被注册成 `knowledge_search` 工具，接入查询改写、并行召回、去重、重排、缓存、熔断和 fallback。

12. 知识检索是用 embedding 模型做的吗？

 是的，知识库检索本质上是 embedding 向量检索。

不过当前代码没有手动调用 Anthropic embedding API，而是使用 ChromaDB 的能力。导入文档时传 `documents`，查询时传 `query_texts`，由 ChromaDB 内部负责生成向量并做相似度检索。

所以准确说，知识检索是基于 embedding 的，但 embedding 生成由 ChromaDB 默认机制处理。

另一个版本

是的，知识库检索本质上是向量检索，会用 embedding 把文档和查询转换为向量，再做相似度匹配。

但需要说明的是，当前知识库检索不是直接调用 Anthropic embedding。代码里使用的是 ChromaDB 的 `add(documents=...)` 和 `query(query_texts=...)` 接口，让 ChromaDB 内部负责 embedding 和向量检索。

在 KnowledgeBase 的实现里：

- 导入文档时传入 `documents`，ChromaDB 自动生成文档向量。
- 查询时传入 `query_texts`，ChromaDB 自动生成查询向量并做相似度检索。

所以面试时可以说：知识库检索是基于 embedding 的向量检索，但 embedding 生成由 ChromaDB 默认机制处理，而不是业务代码手动调用外部 embedding API。

13. 召回以后，所有召回结果都会给到最后的生成模型使用吗？

- 📌 不会。召回结果会先合并去重，再经过 LLM 重排，最后只取 Top-K。
在 `/chat` 主链路里，默认只取前 3 条知识库结果放入 Agent 上下文，而且每条内容会截断到约 600 字。
这样做是为了控制 prompt 长度，减少无关内容干扰，让最终生成模型只看到最相关的知识片段。

另一个版本：

不会。

系统会先通过查询改写产生多个子查询，然后每个子查询并行召回若干结果。多个子查询的结果会合并去重，再经过 LLM 重排，最后只取 Top-K。

在 `/chat` 主链路中，默认只取前 3 条知识库结果拼进 Agent 上下文，并且每条内容最多截取约 600 字。这样做是为了控制上下文长度，避免把太多不相关或弱相关内容塞给最终生成模型。

流程是：

代码块

```
1  多个子查询召回
2    -> 合并去重
3    -> LLM 重排
4    -> 取 Top-K
5    -> 拼成 [知识库检索结果]
6    -> 注入 Agent context
```

14. 用大模型做重排序的目的是什么？



用大模型重排序的目的是提高最终给 Agent 的上下文质量。

向量检索擅长找语义相近的片段，但“语义相近”不一定等于“最能回答用户问题”。例如用户问“退款多久能到账”，向量检索可能同时召回“退款政策”“退货运费”“订单状态”等内容，其中只有一部分真正直接回答到账时间。

LLM 重排会基于原始用户问题，对所有候选片段重新判断相关性，然后给出更适合回答问题的排序。这样最终进入生成模型的 Top-K 片段更精准，可以减少无关上下文干扰。

简而言之，向量检索负责召回，LLM 重排负责精选。

15. 如果大模型本身能判断文档相关性，为什么不直接把所有检索内容都给最终生成模型？



主要有四个原因。

第一，上下文窗口和成本有限。把所有召回内容都放进去会增加 token 消耗和延迟。

第二，过多弱相关内容会干扰生成。大模型虽然能判断相关性，但如果上下文里混入很多噪声，它可能引用错误片段，导致回答不聚焦。

第三，最终生成模型的职责应该是回答用户问题，而不是承担大量检索结果筛选工作。把筛选前置到重排阶段，可以让生成阶段输入更干净。

第四，系统可控性更好。通过 Top-K、截断长度和重排策略，可以明确控制进入最终 prompt 的内容规模。

所以这里采用的是“先召回一批候选，再重排精选 Top-K，再交给生成模型”的架构，而不是把所有候选文档都塞进最终 prompt。

另外一个精简版本：



主要是成本、延迟和噪声问题。

如果把所有召回内容都塞给最终模型，token 会变多，延迟和成本都会上升。而且弱相关内容太多时，模型可能引用错误片段，导致回答不聚焦。

所以我把筛选前置到重排阶段，只把 Top-K 高相关内容交给生成模型。这样上下文更干净，结果也更稳定。

16. 记忆和评测闭环这一块可以介绍一下吗？



记忆闭环是指每次对话都会先读取历史记忆，再把本轮对话写回去。系统会读取 Redis 工作记忆、ChromaDB 情景记忆和用户画像，把它们拼进 Agent 上下文。回复生成后，再写入用户消息和助手回复，并异步更新用户画像。

评测闭环是指系统能自动评估效果。`/eval/run` 会评测意图识别的 Accuracy 和 Macro-F1，也会真实调用 Orchestrator 生成回复，再用 LLM-as-Judge 打分。

评测结果还会和历史基线对比，如果指标明显下降，会标记 regression 并给出优化建议。

17. 三层记忆架构是怎么设计的？它们分别的目标是什么？



三层记忆在 `memory/conversation_memory.py`，分别是工作记忆、情景记忆和用户画像。

第一层是工作记忆，存储在 Redis。目标是保存当前会话最近消息，读写快，适合多轮对话即时上下文。它有 24 小时 TTL，默认最多读取 20 条。

第二层是情景记忆，存储在 ChromaDB 的 `episodic collection`。目标是保存跨会话的历史对话摘要。当 Redis 工作记忆超过阈值时，系统会把旧消息压缩成摘要，并写入情景记忆，后续可以按语义检索回来。

第三层是用户画像，存储在 ChromaDB 的 `user_profile collection`。目标是沉淀用户偏好和关键实体，例如用户常问的问题类型、关注的产品、偏好等。它不是逐字保存聊天记录，而是用 LLM 从对话中提炼结构化信息。

三层目标不同：

- 工作记忆：解决当前会话“刚才说了什么”。
- 情景记忆：解决跨会话“以前发生过什么相关事情”。
- 用户画像：解决长期层面“这个用户有什么偏好和特征”。

18. 三层记忆分别的输入、输出是什么？它们是怎么嵌入到最后生成 Agent 中的？



工作记忆的输入是每轮用户消息和助手回复，输出是最近对话，放到 prompt 的 `[最近对话]` 部分。

情景记忆的输入是被压缩后的历史对话摘要，输出是按当前 query 检索出来的相关历史，放到 `[相关历史]` 部分。

用户画像的输入是近期对话中提炼出的偏好和实体，输出是用户画像 JSON，放到 `[用户画像]` 部分。

这些内容会由 `MemoryContext.to_prompt_text()` 拼成文本，再和 RAG 知识库结果合并成 `full_context`，传给 Orchestrator，最后作为背景信息进入 Agent 的 LLM 调用。

19. 评测用了 LLM-as-judge，评测维度有哪些？

🏆 评测框架在 `evaluation/evaluator.py`，入口是 `/eval/run`。它包含两部分：意图识别评测和端到端回复质量评测。

意图识别评测的指标是：

- Accuracy：预测意图和标注意图一致的比例。
- Macro-F1：按类别计算 F1 后取平均，避免只看总体准确率。

端到端回复质量使用 LLM-as-Judge，维度有四个：

- relevance：相关性，判断回复是否直接回答用户问题。
- accuracy：准确性，判断信息是否正确。
- completeness：完整性，判断是否完整覆盖用户需求。
- helpfulness：有用性，判断用户是否能根据回复采取行动。

每个维度是 0 到 1 分，系统会计算整体均分，并和阈值比较判断是否通过。评测报告还会和历史基线对比，如果某个指标下降超过 5%，会标记为 regression，并给出优化建议。

这里的端到端评测不是 mock 回复，而是真实调用 `AgentOrchestrator.run()` 生成答案，再交给 LLM Judge 评分，所以能更接近实际用户请求链路。

20. 你是在里面做了一个混合检索，然后 BM25 加向量检索加 LLM rerank，讲一下这三部分分别做了什么？

🐒 我这个项目的检索主链路是 RAG 知识库检索，当前代码里已经实现的是 ChromaDB 向量检索，加上 LLM 查询改写、多路并行召回、合并去重和 LLM rerank。

按混合检索的思路来拆，可以分成三层：

第一层是关键词召回，比如 BM25，适合处理订单号、错误码、发票、退款这类强关键词问题。它的优势是精确、可解释，对专有名词和短文本很稳。

第二层是向量检索，也就是当前 EchoMind 已经实现的 ChromaDB 检索。文档导入时切成 chunk，ChromaDB 自动生成 embedding；查询时用 `query_texts` 做语义匹配，解决用户表达和文档措辞不完全一致的问题。

第三层是 LLM rerank，它不是负责扩大召回，而是对已经召回的候选片段按“能不能回答用户问题”重新排序，最后只取 Top-K 注入到 Agent 上下文里。

21. 最后的结果是怎么合并的

 流程：用户原始问题先交给 LLM 做查询改写，生成多个不同角度的子查询。比如“退款多久能到账”，可能改写成“退款审核需要多久”“退款到账需要几天”“退款政策是什么”。

然后每个子查询并行调用 `knowledge_search` 工具，从 ChromaDB 里召回候选文档。召回回来以后，系统不会直接拼在一起给模型，而是先按内容哈希做去重，避免多个子查询召回同一个 chunk。

去重后的候选列表再交给 LLM rerank。rerank 会根据原始问题返回一个相关性排序索引，系统按这个顺序重排，最后截取 Top-K。 `/chat` 主链路默认取前 3 条，拼成 `[知识库检索结果]`，放进 Agent 的背景信息里。

所以合并不是简单拼接，而是：多查询召回、内容去重、统一重排、截断 Top-K。

22. 这个 LLM rank 是用大模型去做，那你这个大模型指的是什么？是 chat 模型，还是 embedding 模型，还是什么模型？

 这里的 LLM rerank 用的是 chat/completion 类模型，不是 embedding 模型。

EchoMind 里 `MCPToolManager` 初始化时会读取环境变量里的 `ANTHROPIC_MODEL`，默认是 `claude-3-5-sonnet-20241022`。如果配置了兼容 Anthropic 协议的 `ANTHROPIC_BASE_URL`，比如 DeepSeek 兼容接口，那 rerank 用的就是配置里的 chat 模型，比如 `deepseek-chat`。

embedding 模型负责把文本变成向量，用于向量检索；而 rerank 需要理解“这个片段是否真正回答了用户问题”，所以我用 chat 模型做判断。实现上就是把用户 query 和候选结果列表放进 prompt，让模型返回按相关性排序的索引数组。

23. 这里的rerank怎么实现的

 实现主要在 `mcp/tool_manager.py` 的 `_rerank()`。

它先把候选结果序列化成编号列表，比如 `0. {...}`、`1. {...}`、`2. {...}`，每个候选会截取前 200 个字符，防止 prompt 太长。

然后 prompt 会要求模型根据用户查询对这些检索结果按相关性排序，只返回 JSON 数组，比如 `[2, 0, 1]`。系统解析这个数组以后，用索引重排原始候选列表，再取前 Top-K。

如果 LLM rerank 调用失败，比如模型超时或者 JSON 解析失败，系统不会让主链路失败，而是 fallback 到原始召回顺序，直接返回前 Top-K。这是为了保证客服系统可用性。

24. 这个模型是你本地部署的，还是说利用 API 这种方式？

 我没有在本地部署 rerank 模型，主要是因为这个项目定位是企业客服系统工程链路，重点在 RAG、Agent 编排、记忆、监控和评测闭环。如果要生产化降成本，可以把 rerank 替换成本地 cross-encoder 或 bge-reranker，但当前代码实现是 API 调用 chat 模型 rerank。

25. 你的这个 Top-K 这些参数是你自己调的，还是有什么策略？主要靠体感吗，还是有没有做评测？

 不是只靠体感。当前代码里有两个 Top-K：召回阶段和最终注入阶段。

在 `search_with_rewrite()` 里，召回阶段会取 `recall_k = max(top_k, 5)`，也就是说即使最终只要 3 条，也会至少先召回 5 条，给 rerank 留一点候选空间。

在 `/chat` 主链路里，`_build_knowledge_context()` 默认最终注入 Top-3。这个选择主要是工程折中：Top-1 可能漏信息，Top-5 或更多会增加 token、延迟和噪声；Top-3 对客服知识库这种短政策文档比较稳。

评测方面，项目里已经有 `/eval/run`，可以真实调用 Orchestrator 生成回答，再用 LLM-as-Judge 从相关性、准确性、完整性、有用性四个维度评分。更严格地调 Top-K，我会固定一批问答集，对 Top-1、Top-3、Top-5 分别跑评测，看准确性和完整性有没有提升，同时观察延迟和 token 成本。

所以当前 Top-K 是基于工程经验设置，并且项目已经提供了评测框架，可以进一步系统化调参。

26. 你了解 RAG 这种知识问答的评测体系吗？比如说你用什么方法可以评测你的这个系统，做到回答的质量？

 我会把 RAG 评测拆成两层：检索层和生成层。

检索层看召回的知识片段是否对。常见指标有 Recall@K、Precision@K、MRR、NDCG。比如用户问“退款多久到账”，如果标准证据是“退款政策”这个 chunk，那么这个 chunk 是否出现在 Top-K 里，就是召回质量。

生成层看最终回答是否好。EchoMind 当前已经实现的是 LLM-as-Judge 评测，在 `evaluation/evaluator.py` 里。它真实调用 `AgentOrchestrator.run()` 生成回答，然后让 Judge 模型从四个维度打分：`relevance` 相关性、`accuracy` 准确性、`completeness` 完整性、`helpfulness` 有用性。

此外还有意图识别评测，计算 Accuracy 和 Macro-F1。评测报告会保存 baseline，下一轮如果指标下降超过 5%，会标记 regression，并给出优化建议。

如果进一步完善 RAG 评测，我会加两类用例：第一类是带标准答案和标准证据的知识问答集，用来算检索 Recall@K 和答案 faithfulness；第二类是业务场景用例，比如退款、发票、登录报错、重复扣款，评估端到端回答是否引用正确政策、是否给出可执行步骤。

27. Agent Orchestrator 是什么，真值来源是什么？



`AgentOrchestrator` 是多 Agent 编排器，不是真值来源本身。

它负责把一次用户请求路由到合适的 Agent。流程是：先做意图识别，然后根据意图映射到 `TechnicalAgent`、`BillingAgent` 或 `GeneralAgent`；如果一句话同时涉及技术和账单，比如“登录报错 401，而且重复扣款”，它会并行调用技术和账单两个 Agent，再把结果合并。

真值来源主要是知识库和对话上下文。业务规则来自 ChromaDB 里的 `knowledge_base`，比如退款政策、订单查询、账户安全、技术故障排查、会员积分、配送说明。用户历史来自 Redis 工作记忆、ChromaDB 情景记忆和用户画像。

所以 Orchestrator 的职责是调度和路由，真正约束回答内容的是 RAG 知识库、记忆上下文和各个 Agent 的 system prompt。为了减少幻觉，`/chat` 会把 Top-K 知识库检索结果拼到背景信息里，并提示 Agent 优先依据知识库回答。

28. 如果不同的 agent 能回答不同的知识的话，那你这个不同的知识一开始是怎么区分的？比如账单问题一个文档，技术问题一个文档，还是一个文档里都有？



当前 EchoMind 里是一个统一的 `knowledge_base` collection，不是每个 Agent 一个独立知识库。默认文档里有退款政策、订单查询、账户安全、技术故障排查、会员积分、配送说明等内容。

知识本身没有强制按 Agent 拆库，路由和知识检索是两条链路。`IntentRecognizer` 负责判断问题应该交给哪个 Agent，比如 technical 或 billing；RAG 检索负责从统一知识库找相关证据。

这样做的好处是简单，而且能处理跨域问题。比如“支付失败”既可能是账单问题，也可能是技术问题，如果强行拆库可能漏召回。

如果后续业务文档很多，我会在 metadata 里加 `domain` 字段，比如 `billing`、`technical`、`account`。检索时可以先全局召回，再根据意图加权，或者对 BillingAgent 只过滤 billing/account 文档，对 TechnicalAgent 过滤 technical 文档。这样既保留统一知识库，又能做 Agent 级权限和领域过滤。

29. 你是怎么识别用户意图的？



EchoMind 的意图识别在 `core/intent_recognizer.py`，用的是三路融合。

第一路是 LLM 语义理解。它用 few-shot 示例和最近 3 轮对话上下文，让模型输出 intent、confidence 和 reasoning。这个负责复杂表达，比如用户说“钱扣了但是服务打不开”，LLM 能理解里面同时有账单和技术含义。

第二路是 Embedding 相似度。系统把用户问题和每类意图模板做相似度匹配。（当前 Anthropic SDK 没有 embedding 资源，所以代码做了远端优先、本地字符 n-gram 哈希向量兜底；如果配置第三方兼容 API，默认禁用 embedding 分支，避免接口不兼容。）

第三路是关键词 Pattern，比如“退款、扣款、发票”倾向 billing，“报错、崩溃、error”倾向 technical，“转人工、经理、投诉”倾向 escalation。

最后通过加权投票融合。官方模式下是 LLM 70%、Embedding 20%、Pattern 10%；第三方兼容 API 模式下是 LLM 85%、Pattern 15%。得分低于阈值就降为 `OTHER`，减少错误路由。

30. 追问29问题：这里的 LLM 70%、Embedding 20%、Pattern 10%比例为什么这么设置

补充：回答的时候可以说是按 Accuracy、Macro-F1 和各类意图的 Precision/Recall 来调权重，比如对退款、技术、转人工这些高频类别分别看误判情况，再做网格搜索或自动调参得到的)



这个权重不是训练出来的参数，而是我在工程上根据三路信号的可靠性做的初始配置。

LLM 权重最高，是因为它最能理解复杂语义和上下文。比如用户说“钱扣了但是服务打不开”，这里既有账单含义，也有技术含义，单靠关键词或者模板相似度很容易判断不完整，所以官方模式下我给 LLM 70%。

Embedding 给 20%，主要作为语义相似度辅助。它适合处理一些常见表达和模板匹配，比如“申请退款”和“钱什么时候退回来”表述不同，但语义接近。它不作为主判断，是因为模板覆盖有限，而且相似度高不一定代表业务意图完全正确。

Pattern 给 10%，因为关键词规则精确但比较脆。比如出现“退款”“扣款”“发票”时，对 billing 很有提示作用；出现“报错”“崩溃”“error”时，对 technical 很有提示作用。但是关键词不理解上下文，容易误伤，所以我把它作为兜底信号，而不是主信号。

第三方兼容 API 模式下，很多接口不支持 embedding，所以我把 Embedding 这一路禁用。禁用后权重不能空着，就转移给 LLM 和 Pattern，变成 LLM 85%、Pattern 15%。这里 Pattern 提高到 15%，是为了保留对高置信关键词的兜底能力，比如“退款”“转人工”“发票”这类客服场景里的强业务词。

得分低于阈值降级为 OTHER，是为了避免低置信度时强行路由到错误 Agent。客服系统里错误路由的代价比较高，比如账单问题错路由到技术 Agent，回答就会偏。所以我宁愿在不确定时降级，再由 GeneralAgent 兜底或引导用户补充信息。

31. 用户问题同时涉及账单和技术怎么办？



这个场景 EchoMind 已经在 Orchestrator 里处理了。

意图识别通常输出一个主意图，但

`AgentOrchestrator._collaboration_targets()` 会额外检查领域关键词。如果消息里同时出现技术关键词和账单关键词，比如“登录报错 401，而且这个月重复扣款了”，系统会识别出 TechnicalAgent 和 BillingAgent 两个目标。

然后 `run_parallel()` 会并行执行多个 Agent。技术 Agent 负责登录报错、401、排障步骤；账单 Agent 负责重复扣款、退款、人工审核建议。最后 Orchestrator 把两个成功响应合并，返回给用户。

这样比只选一个 Agent 更稳，因为跨领域问题如果只路由到账单，可能漏掉技术排查；只路由到技术，又可能漏掉退款和账单处理。

32. 你这个 agent 我看还有 agent orchestrator，它是干嘛的？你能讲一下你为什么要搞这个吗？



我做 Orchestrator 是为了解决多 Agent 系统里的调度问题。

如果只有一个 Agent，所有问题都塞给同一个 prompt，短期可以跑，但技术问题、账单问题、投诉升级的处理方式其实不一样。技术问题需要排障步骤，账单问题要谨慎处理金额、退款、发票，升级问题要标记人工介入。

所以 Orchestrator 做三件事：

第一是路由。根据 `IntentRecognizer` 的结果，把请求发给 Technical、Billing 或 General Agent。

第二是协作。一个问题如果同时涉及多个领域，就并行调用多个 Agent，再合并结果。

第三是可靠性。目标 Agent 不存在或调用失败时，降级到 GeneralAgent；如果紧急度高或用户要求转人工，就设置 `escalated=true`。

这个模块的价值是把“谁来回答、怎么协作、失败怎么办”从具体 Agent 里抽出来，后续扩展更多 Agent 时不会把主链路写乱。

33. 为什么不同 Agent 不只是换提示词？

⚽ 当前实现里，几个 Agent 的底层调用方式是一致的，主要差异确实是职责边界和 system prompt。比如 TechnicalAgent 的 prompt 要求给故障排查步骤，BillingAgent 要求对退款、发票、订阅保持准确和谨慎。

但我做多 Agent 的目的不只是换 prompt，而是为后续工程扩展留接口。

因为拆成 Agent 后，每个 Agent 可以有独立工具、独立知识过滤、独立权限、独立模型和独立监控指标。比如账单 Agent 将来可以接订单系统和退款工具，技术 Agent 可以接日志查询和故障诊断工具，投诉 Agent 可以接工单系统。

当前版本先把路由、并行协作、降级和监控统计搭起来，后续增强每个 Agent 的工具能力会比较自然。

34. 如果知识库和大模型自己的常识冲突，怎么办？

🦄 在客服系统里，我会让业务知识库优先级高于模型常识。

EchoMind 在 `_build_knowledge_context()` 里会把检索到的 Top-K 片段拼成 [知识库检索结果]，并明确提示 Agent：“请优先依据以上知识库内容回答；如果知识库内容不足，再结合通用客服能力说明。”

所以比如知识库写退款审核是 1-3 个工作日，模型不能凭常识说“当天到账”。如果知识库没覆盖，Agent 才可以给通用建议，并提示用户联系人工确认。

评测上也应该检查忠实度，也就是回答是否和检索证据一致。这个可以通过 LLM-as-Judge 加一个 faithfulness 维度，或者人工抽样检查高风险业务问题。

35. 如果检索不到内容，或者知识库挂了怎么办？

❤️ 有两层处理。

第一，如果检索没有结果，`_build_knowledge_context()` 会返回空知识上下文，主链路仍然会继续走 Agent 回复，只是这次 `knowledge_used=false`。Agent 会基于通用客服能力和记忆回答。

第二，如果知识库工具异常，比如 ChromaDB 不可用，`knowledge_search` 注册了 fallback。fallback 会返回一条“知识库暂时不可用”的可解释降级结果，而不是让整个 `/chat` 崩掉。

工具框架本身还有熔断器，连续失败到阈值后会短期开启熔断，避免所有请求都卡在故障工具上。这个设计是为了保证客服主链路可用。

36. 为什么要做 query rewrite，直接搜不行吗？



直接搜可以，但单一 query 容易召回不全。

用户表达通常很短，比如“钱什么时候回来”。如果直接搜，向量检索可能不知道这是退款到账问题。查询改写会把它扩展成更明确的搜索子查询，比如“退款到账时间”“退款审核流程”“原路退回需要多久”。

EchoMind 的 `rewrite_query()` 会保留原始 query，同时生成多个角度的子查询。然后多个子查询并行召回，最后合并去重和 rerank。这样既扩大召回覆盖面，又不会明显增加串行延迟。

如果 LLM 查询改写失败，系统会 fallback 到原始 query，不影响主链路。

37. 既然最后也是大模型判断，为什么不把所有召回内容都塞给最终生成模型？



主要是三个原因：成本、延迟和噪声。

召回内容越多，prompt token 越多，接口成本和响应延迟都会上升。更重要的是，弱相关内容太多会干扰模型，模型可能引用错误政策，导致回答不稳定。

所以 EchoMind 把筛选前置到 rerank 阶段。先用多个子查询扩大召回，再用 LLM rerank 精选 Top-K，最后只把最相关的 3 条放到 Agent 背景里。

这相当于让最终生成模型看到更干净的上下文，回答会更聚焦。

38. 你怎么证明回答质量不是靠体感？

 不是只靠体感。EchoMind 有专门的评测入口 `/eval/run`。

它包含两类评测：第一类是意图识别评测，输入带标注的测试用例，输出 Accuracy 和 Macro-F1；第二类是端到端对话评测，系统会真实调用 `AgentOrchestrator.run()` 生成回复，再用 LLM-as-Judge 从相关性、准确性、完整性、有用性四个维度打分。

评测报告还会保存 baseline，如果下一次指标下降超过 5%，会输出 regression。这样可以用于改 prompt、改检索参数、改 Agent 路由后的回归检查。

当前内置用例规模不大，所以我不会说它已经证明线上准确率很高。我会说项目已经具备自动评测框架，后续只要补充标注用例，就能系统化评估，而不是靠人工感觉。

39. 先从一次用户请求开始，把完整线上链路讲清楚

 可以，我按一次用户提问来讲。用户消息进入服务后，系统先拿到用户问题、用户标识和会话标识；如果没有会话标识，就新建一个，方便后续多轮对话追踪。

第一步是读取记忆。系统会取当前会话最近的对话、历史相关对话、用户画像和会话摘要，把这些内容整理成模型可以理解的背景信息。

第二步是判断要不要查知识库。像“你好”这种寒暄不会查知识库；如果是订单、退款、发票、登录、报错、支付这类业务问题，就会进入知识库检索。检索时会先把用户问题改写成几个不同角度的问题，再并行召回知识片段，去重后重新排序，最后取最相关的几条放进上下文。

第三步是意图识别和智能体路由。系统会结合大模型判断、语义相似度和关键词规则，判断用户是普通咨询、技术问题、账单问题、投诉还是转人工。然后根据意图选择通用客服、技术支持或账单客服。如果一句话同时涉及技术和账单，比如“登录报错还被重复扣款”，系统会让多个智能体并行处理，再合并回答。

第四步是生成回答。每个智能体都有自己的角色设定，同时会按当前问题匹配对应的业务能力包，比如退款规范、技术排障规范、通用客服规范。模型会结合记忆、知识库结果、能力包规则 and 用户问题生成最终回复。

最后，系统把用户问题和助手回答写回短期记忆，并异步更新用户画像。接口返回时会带上回答内容、识别到的意图、使用的智能体、是否使用知识库、是否需要升级人工，以及本次

请求耗时。

40. RAG 当中的数据怎么处理，完整流程是什么，分块策略怎么选，有没有考虑 overlap

知识库数据先通过后台接口或文件上传进入系统。进入后，系统会把每篇文档切成多个片段，每个片段记录标题、片段序号和总片段数，然后存入向量数据库。向量化这一步由向量数据库内部完成，项目里没有单独维护一个向量服务。

当前分块策略是每块大约五百字，尽量按句号和换行切分，不做硬切。这样做是因为客服知识库大多是问答、政策和排障说明，五百字通常能覆盖一个完整规则，同时也能控制放进模型的上下文长度。

检索流程是：先判断是否需要查知识库，再做查询改写，然后多路并行召回，接着去重，再用大模型重新排序，最后取最相关的几条给回答使用。

目前项目没有实现片段重叠，这是一个可以优化的点。如果文档变成长制度、长手册或者技术文档，我会给相邻片段加五十到一百字重叠，或者保留上一段最后一两个句子，避免答案刚好被切在两个片段之间。

41. 了解 Skills 吗，Skills 的结构是什么，平时有没有自己写过 Skills

我了解 Skills。我的理解是，Skills 本质上是把某一类稳定的业务能力、操作规范、领域知识或者工具使用流程，从代码里抽出来，做成可加载的指令包。它不是一次性的 prompt，而是可以被复用、被检索、被按场景注入的能力模块。

在这个项目里，Skills 的结构比较轻量。标准形式是 `skills/某个目录/SKILL.md`。每个 `SKILL.md` 顶部有 front matter，里面定义 `name`、`description`、`keywords`、`agents`、`enabled`。正文就是具体的业务规则。比如账单退款 Skill 只给 BillingAgent 用，关键词包含退款、扣款、发票、订阅；技术支持 Skill 只给 TechnicalAgent 用，关键词包含报错、错误、API、SDK、500、401 等。

项目里也支持其他结构，比如 `skills/refund.json`、`skills/refund.md`、`skills/refund.txt`。JSON Skill 里可以写 `content` 或 `instructions`，Markdown Skill 可以用 front matter。加载后都会标准化成 `Skill` 对象，字段包括 `name`、`description`、`content`、`path`、`keywords`、`agents`、`enabled`。

运行时的逻辑是：用户请求进来后，Agent 在构造 system prompt 时会调用

`SkillManager.prompt_for(message, agent_type)`。SkillManager 会先过滤 disabled 的 Skill，再看 agents 是否匹配当前 Agent，再看 keywords 是否命中用户消息。命中的 Skill 会被格式化成 prompt block，拼入 system prompt。为了避免上下文过大，项目里还设置了 `ECHOMIND_SKILLS_MAX_PROMPT_CHARS`，默认最多注入 5000 字。

我自己写过类似 Skills。比如这个项目里的账单退款处理规范、通用客服接待规范、技术支持处理规范，本质上就是把客服 SOP 变成可热加载的 Skills。这样运营侧改话术、改规则，不一定要改 Agent 代码，只要更新 Skill 文件，然后调用 `/skills/reload` 就能生效。

42. 平时有没有使用过 AI 编程工具，对 AI 编程工具的理解是什么



我平时会使用 AI 编程工具。我的理解是，AI 编程工具不是简单的代码补全，而是一个“带工具调用能力的工程代理”。它的价值不只是生成一段代码，而是能读项目上下文、定位文件、理解约束、执行测试、根据报错迭代修复，并把改动控制在合理范围内。

我会把 AI 编程工具分成三层能力。第一层是代码理解能力，比如读目录结构、跨文件搜索、理解调用链。第二层是代码修改能力，比如按现有风格改代码、补测试、修文档。第三层是工程闭环能力，比如运行单测、看日志、定位失败原因、做最小修复。

我使用这类工具时不会完全依赖它“凭空写”。更合理的方式是把它当成一个结对工程师：先让它读代码，要求它基于当前项目模式修改；再让它跑验证；最后自己 review diff。尤其是涉及生产配置、鉴权、数据库迁移、缓存、并发和异常处理时，我会特别关注它有没有扩大修改范围、有没有遗漏边界条件。

结合 EchoMind 这个项目，AI 编程工具最适合做的事情包括：梳理调用链、修改模型配置、修缓存顺序这种局部逻辑、补文档、写面试稿、补充评测用例。它不适合直接替我拍脑袋决定产品策略，比如退款规则怎么变、人工升级标准怎么定，这些需要业务输入。

43. Claude Code 的原理了解过吗，你的 Skills 加载机制是什么



我了解一些 Claude Code 的工作方式。我的理解是，它本质上也是一个 Agentic Coding 工具：模型在一个代码仓库环境里工作，通过工具读取文件、搜索代码、编辑文件、执行命令，然后根据执行结果继续推理和修改。它不是只看当前打开的一个文件，而是通过工具逐步建立项目上下文。

Claude Code 的上下文机制里，比较典型的是项目级记忆文件，比如 `CLAUDE.md`。这类文件会放项目约定、运行命令、测试方式、代码风格、注意事项，让每次会话启动时都能带上仓库规则。它还支持 Skills 这种能力包，Skill 通常包含 `SKILL.md`，里面写这个能力什么时候使用、怎么使用，必要时还可以带脚本和参考资料。和普通全量 prompt 不一样，Skills 更强调按需加载，避免把所有长文档一次性塞进上下文。

我这个项目里的 Skills 加载机制更简单，是我自己实现的一套业务规则注入，不是 Claude Code 原生 Skills。启动时 `SkillManager` 会扫描 `ECHOMIND_SKILLS_DIR` 指定目录，优先读取所有 `SKILL.md`，也支持 `.json`、`.md`、`.txt`。解析后形成 `Skill` 对象，存到内存里。每次 Agent 调 LLM 前，根据当前消息和 Agent 类型选择匹配的 Skill，拼到 system prompt 的 `[动态 Skills]` 段落里。

这里还有两个工程点。第一，Skills 支持热加载，`POST /skills/reload` 会重新扫描目录，并更新 Orchestrator 里每个 Agent 引用的 SkillManager。第二，Skills 有长度控制，避免业务规则太多导致上下文膨胀。这个实现的目标不是做一个通用 IDE Agent，而是服务客服 Agent 的业务规则治理。

44. Claude Code、Trae、Codex 的 Skills 加载机制了解过吗，和你的有什么区别



我会把它们分成两类来比较：一类是编程工具里的 Skills，另一类是这个项目里的业务 Skills。

Claude Code 和 Codex 这类编程工具的 Skills，通常是面向“如何完成某类工程任务”的。比如生成图片、写测试、做代码审查、调用某个外部 API，它们会包含说明文件、脚本、参考资料、模板或资产。加载机制通常是渐进式的：基础规则会常驻，例如项目级的 `CLAUDE.md` 或 `AGENTS.md`；具体 Skill 的详细内容只在模型判断需要时加载，或者用户显式点名时加载。这样做是为了节省上下文窗口，同时让工具在需要时能拿到完整操作步骤。

Codex 的 Skills 也有类似结构，一个 Skill 目录里核心是 `SKILL.md`，还可以带 `scripts`、`references`、`assets` 等资源。我的理解是，`SKILL.md` 负责告诉模型这个 Skill 什么时候触发、怎么执行；脚本用于把复杂操作固化成可运行工具；参考资料用于按需读取，避免所有资料常驻上下文。Codex 还会读取项目里的 `AGENTS.md`，作为仓库级协作规则。

Trae 这类 IDE Agent 更强调 IDE 内上下文和项目规则，比如项目索引、打开文件、用户选区、终端输出、规则文件等。不同版本具体叫法可能不同，但核心思想也是让模型结合工程上下文和用户规则工作。

和它们相比，EchoMind 的 Skills 更轻量、更业务化。我的 Skills 不负责操作文件、运行脚本或驱动 IDE，而是负责给客服 Agent 注入业务 SOP。加载方式也更明确：启动时扫描目录，运行时根据 `agents` 和 `keywords` 匹配，然后把正文拼进 system prompt。它没有做复杂的语义检索匹配，也没有脚本执行能力。优点是简单、可控、容易热更新；缺点是触发方式主要依赖关键词，复杂意图下可能需要结合 embedding 或 LLM 分类进一步优化。

45. 查询重写有必要吗，你们查询重写是怎么做的



有必要，特别是在客服场景。用户经常用口语表达，比如“钱什么时候回来”，但知识库可能写的是“退款到账时间”“原路退回”“支付渠道处理时效”。如果直接检索，召回不一定稳定。

我们会用大模型把原始问题改写成几个不同角度的检索问题，同时保留用户原始问题。然后多个问题并行检索，召回结果合并去重，再按原始问题重新排序。

这里的关键是控制改写不要跑偏。改写只能扩展表达，不能改变用户意图。如果改写失败，就退回到只用原始问题检索。

46. 短期记忆和长期记忆模块怎么做，存储策略是什么，Redis key-value 结构是什么，工程上怎么落地



这个项目的记忆模块是三级记忆。

第一层是短期记忆，也叫工作记忆，用 Redis 存。它保存当前会话最近的多轮消息，解决“刚才说的那个订单”“继续上一个问题”这类上下文连续性问题。Redis key 是 `wm:{user_id}:{conv_id}`，value 是一个 Redis list。每条消息是 JSON 字符串，字段包括 `role`、`content`、`ts`、`metadata`。写入时用 `lpush`，所以最新消息在列表前面；读取时用 `lrange(key, 0, WORKING_MAX - 1)`，然后 `reversed` 还原时间顺序。这个 key 设置 24 小时 TTL。

第二个 Redis key 是摘要 key，格式是 `summary:{user_id}:{conv_id}`。当工作记忆超过压缩阈值时，系统会把旧消息总结成摘要，存到这个 key 里，也设置 24 小时 TTL。这样当前会话的长历史不会全部塞进 prompt。

第二层是长期的情景记忆，用 ChromaDB 的 `episodic` collection 存。它不是保存每条原始消息，而是当工作记忆压缩时，把被压缩的旧消息总结成 summary，再把 summary 作为 document 写入 ChromaDB。metadata 里有 `user_id`、`conv_id`、时间戳，以及截断后的 `full_text`。下次用户提问时，系统会用当前 query 到 `episodic` 里按 `user_id` 过滤做语义检索，取最相关的历史片段。

第三层是用户画像，也用 ChromaDB，collection 叫 `user_profile`。每轮对话返回后，系统会异步调用 `update_profile()`，从最近 10 条对话里提取用户偏好和关键实体，要求 LLM 返回 JSON，比如 `preferences`、`entities`。然后把这个 JSON 文本写入 ChromaDB。后续构建上下文时会取用户画像，拼到 prompt 里。

工程落地上，读取链路是 `get_context()`：先读 Redis 工作记忆，再检索 ChromaDB 情景记忆，再读 ChromaDB 用户画像，再读 Redis 摘要，最后格式化成 prompt。写入链路是 `add_message()`：用户消息和助手回复都写 Redis；如果列表长度超过 15 条，触发 `_compress()`；接口返回后异步更新画像。

这个设计的核心是把不同生命周期的数据放到不同存储里。短期、频繁读写、强时序的数据放 Redis；长期、需要语义检索的数据放 ChromaDB；用户画像作为长期偏好单独存，避免和普通历史对话混在一起。

47. 有没有考虑过上下文窗口爆炸

 有。项目里做了几层限制：短期记忆有最大条数，超过阈值会压缩；知识库只取最相关的几条，每条内容也会截断；能力包有最大注入长度；意图识别只看最近几轮历史。

如果继续优化，我会做更细的上下文预算。比如系统规则优先级最高，其次是命中的能力包、知识库高相关片段、最近对话和历史摘要。这样在上下文不够时，可以按优先级裁剪，而不是简单截断。

48. 线上用户反馈“问 A 回 B”或始终得不到理想答案，怎么排查，哪些因素有影响

 我会按链路排查，而不是直接认为是模型问题。先复现用户请求，看用户标识、会话标识、原始问题、识别意图、选择的智能体，以及这次是否用了知识库。

然后查知识库链路：该查的时候有没有查，查询改写有没有跑偏，召回片段是否相关，排序是否合理，知识库内容是否过期。再查意图和路由：账单问题有没有被分到技术支持，复合问题有没有触发多个智能体。

接着查能力包和记忆：有没有命中错误能力包，关键词是不是太宽；短期记忆、会话摘要、用户画像有没有污染回答。最后再看工具缓存、熔断、降级策略、模型配置和日志。

修复要按原因来。意图错就补样本和关键词；知识库错就改文档、分块、重叠、改写和排序；能力包错就收窄关键词；记忆污染就清理或优化摘要。线上还可以结合监控和评测，把问题样本沉淀成回归用例。

49. 如果大模型没有找到相关问题，会怎么回答？用户投诉回答不正确怎么排查？有专门日志吗

 如果知识库没有召回有效内容，系统不会强行编答案，而是回到通用客服能力和 Skills 规则来回答。比如能解释通用流程就解释；如果涉及订单状态、退款结果、账户权限这类需要真实系统核验的问题，会提示用户补充必要信息，或者建议转人工处理。

用户投诉回答不正确时，我会先按链路排查：原始问题是什么，是否使用了 RAG，召回内容是什么，最终路由到了哪个 Agent，命中了哪些 Skills，短期记忆和用户画像有没有把模型带偏。

日志方面，项目现在有基础日志，比如模型配置、Skills 加载和命中、查询重写、工具异常、知识库失败、Agent 成功率和延迟等。但严格说，目前还不是完整的结构化链路日志。真正上线我会补一个完整 trace，把一次请求的意图、路由、召回片段、重排结果、命中 Skills、最终 Agent 和耗时都记录下来，方便复盘。

50. 向量检索召回了正确文档，但模型却说未找到相关信息，怎么排查和解决？



这种情况说明问题不在召回，而在召回结果到模型回答之间。排查时我会先确认正确文档有没有真的进入最终上下文，而不是只在检索阶段出现。因为中间还有去重、重排、截断、上下文拼接这些步骤，正确片段可能被排到后面，或者被截断掉。

然后看 prompt。模型说未找到相关信息，可能是知识库内容写得不够直接，也可能是系统提示里“知识不足再结合通用能力”这句话让模型过于保守。还有一种情况是记忆或 Skills 和知识库内容冲突，模型优先听了别的上下文。

解决方式是：把知识库结果放到更靠近用户问题的位置；明确要求优先依据知识库回答；回答时尽量引用召回片段里的关键信息；减少无关记忆干扰；必要时提高正确文档的排序权重。线上还要把召回片段和最终 prompt 记录下来，方便复盘。

51. 项目数据入库怎么做？数据清洗怎么做？图片和视频有没有纳入 RAG？



项目目前支持文本类知识入库，可以批量传文档，也可以上传文本、Markdown 或 JSON 文件。入库后系统会按句子边界切分成小片段，给每个片段生成标识，并保存标题、片段序号等元信息，然后写入 ChromaDB。

数据清洗目前比较基础，主要是做文本读取、编码容错、空内容过滤、按句子切分和元信息整理。也就是说，它能处理常见客服知识文档，但还不是完整的数据治理流水线。

图片和视频目前没有纳入 RAG 链路。原因是当前项目定位是客服文本问答，知识库处理的是文本内容。如果后续要支持图片和视频，我会先做内容抽取：图片走文字识别或多模态理解，视频先抽字幕、语音转文字和关键帧说明，最后仍然转成本片段进入知识库。

52. 项目有没有用 LangChain？混合召回、多路检索的优点是什么？

 这个项目没有使用 LangChain 这类框架，核心链路是自己写的。这样做的好处是链路更透明，查询重写、工具调用、缓存、重排、降级都能自己控制；缺点是需要自己维护更多工程细节。

混合召回和多路检索的优点是召回更稳。单一路径容易受用户表达影响，比如用户说法很口语，向量检索可能召回不准。多路检索会把原始问题和多个改写问题一起查，覆盖不同表达方式。

另外，多路召回后再去重和重排，可以把“召回全”和“排序准”拆开处理。先尽量把可能相关的内容捞出来，再用模型判断哪些最适合回答用户问题。

53. 多 Agent 路由解决什么问题？为什么不用一个通用 Agent？线上怎么知道路由到了哪个 Agent？怎么评估？

 多 Agent 路由解决的是专业边界问题。客服场景里，技术排障、账单退款、通用咨询的回答规范不一样。如果只用一个通用 Agent，prompt 会越来越长，规则容易冲突，回答也不够稳定。

项目里把职责拆成通用客服、技术支持、账单客服。好处是每个 Agent 的角色更清晰，Skills 也能按领域注入。比如退款问题要强调核验和资金风险，技术问题要强调错误码、环境和复现步骤。

线上可以从接口返回里看到最终使用的 Agent 类型，也可以从日志和监控里看路由结果、成功率和耗时。评估路由准确率主要靠离线评测集：准备一批标注了正确意图的用户问题，看系统预测的意图和目标 Agent 是否一致。真实上线后，还要把用户投诉和人工纠正样本沉淀回评测集。

54. 如果意图识别错了会怎么样？怎么解决？

 意图识别错了，最直接的影响就是路由错。比如退款问题被分到技术支持，回答就会偏排障；技术问题被分到账单客服，用户也会觉得答非所问。

项目里有几层兜底。第一，关键词规则可以补足一些明显场景。第二，复合问题会再用领域关键词判断是否需要多个 Agent 协作。第三，专属 Agent 失败时可以降级到通用客服。第四，如果识别成高紧急或转人工，会触发升级。

解决方式是把 bad case 加入离线评测集，补充 few-shot 样本和关键词，调整置信度阈值；如果某类问题经常混淆，就单独拆规则或增加实体判断。

55. 当前有哪些 Agent？职责边界怎么划分？

🏠 当前主要有三个 Agent。GeneralAgent 负责通用咨询、问题澄清、简单售后和分流；TechnicalAgent 负责登录失败、报错、接口问题、部署配置、性能和故障排查；BillingAgent 负责退款、扣款、账单、发票、订阅和支付争议。

边界划分的原则是：通用问题先由通用客服接待；涉及错误码、系统异常、接口和配置的交给技术支持；涉及钱、发票、退款和订阅的交给账单客服。如果问题跨多个领域，就并行调用多个 Agent。

56. 高并发下 Redis 或向量数据库变慢，会不会拖垮 Agent 路由？

🚢 有这个风险。因为主链路一开始就要读短期记忆，业务问题还要查向量数据库。如果 Redis 或 ChromaDB 变慢，会直接拉长请求耗时，严重时会导致 Agent 调用前就卡住。

项目里目前有一些缓解：工具调用有超时、熔断、缓存和降级；知识库构建失败时不会让整个对话失败，而是降级为不带知识库上下文继续回答；监控会采集工具成功率和延迟。

如果真要抗高并发，我会继续加几层保护：Redis 连接池和超时控制，向量检索限流，热门查询缓存，知识库失败快速降级，异步画像更新不阻塞主链路，以及按服务拆分扩容。核心原则是记忆和检索变慢时，不能拖垮最终回答链路，最多降低上下文质量。

下面这部分朋友把这个项目包装成运维的面经，先分享给大家

1. 为什么不直接把问题丢给大模型去做运维排查？你设计这一套复杂的 Agent 架构，优势在哪里？
2. 你提到接入了 GitLab 工具，具体是怎么检索的？拉取什么内容？
3. 能不能举一个真实的故障排查案例，说明工具是怎么工作的，最终解决了什么问题？
4. 你刚才举的例子看起来更像代码查询，而不是故障排查。接入这些工具到底是怎么提升故障排查效率的？
5. 你说系统只输出一个主意图，那什么时候会触发多 Agent？
6. 你的意图识别只有三种分类吗？意图识别输出主意图，为什么还命中关键词来触发多 agent
7. 你这套逻辑本质上是不是还是关键词匹配？请拆解一下第一步算完主意图之后，第二步具体做什么？
8. 如果主意图都没命中，说明指令很模糊，这时候触发多 Agent 并行分析合理吗？

9. 既然第二步多Agent的分析能力更强，甚至能处理复合问题，那还要第一步的意图识别干什么？是不是多余的？